QUESTION 1

Water Jug Problem – Step-by-Step Calculation

Given: 4-gallon jug and 3-gallon jug. No markings. Allowed operations: fill, empty, and pour. Goal: exactly 2 gallons in the 4-gallon jug.

State = (4L jug, 3L jug). Initial state = (0,0). Goal = (2, y).

Step	Operation	Calculation	New state
1	Fill 4L	4L jug: 0+4=4; 3L jug stays 0	(4,0)
2	Pour 4L → 3L	3L needs 3–0=3L; 4L left = 4–3=1L	(1,3)
3	Empty 3L	3–3=0; 4L stays 1	(1,0)
4	Pour 4L → 3L	Move the 1L: 4L=1–1=0; 3L=0+1=1	(0,1)
5	Fill 4L	4L: 0+4=4; 3L stays 1	(4,1)
6	Pour 4L → 3L	3L already has 1L, so free space=3–1=2L; 4L gives 2L; 4–2=2	(2,3)

Why Step 6 gives exactly 2 gallons: the 3-gallon jug already contains 1 gallon, so it can accept only 2 more gallons. Therefore exactly 2 gallons leave the 4-gallon jug.

Check	Calculation	Result
Target	4L jug = 2 gallons	✓
No measuring device	Only fill, empty and pour used	✓
All operations legal	Every pour stops when source is empty or destination is full	✓

Final answer: (0,0) → (4,0) → (1,3) → (1,0) → (0,1) → (4,1) → (2,3).

QUESTION 2

Mars Rover – Agent Analysis

The rover must explore Mars, detect and analyse samples, avoid hazards, manage energy and communicate useful scientific results. The answer below gives the required calculation/decision logic without unnecessary theory.

i. Percepts

Sensor / percept	Information obtained
Camera	Rocks, terrain, obstacles and sample locations
Spectrometer	Chemical/mineral composition
Temperature / pressure sensors	Environmental conditions
Navigation sensors	Position, direction and nearby obstacles
Power sensors	Battery and system condition
Sample instruments	Properties of collected samples

ii. Operating environment

Property	Answer	Reason
Observability	Partially observable	Only local surroundings are sensed
Agents	Mainly single-agent	Rover makes local decisions
Determinism	Uncertain	Terrain and measurements may be uncertain
State	Sequential	Each movement affects later choices
Nature	Mostly dynamic	Dust, temperature and system state can change
Knowledge	Partly unknown	New terrain and samples are discovered

iii. Actions

Action	Example
Move	Drive / turn / stop
Observe	Take images and readings
Collect	Pick up rock/soil sample
Analyse	Run scientific instrument
Communicate	Transmit important data
Power management	Save energy / enter safe mode

iv. Performance calculation

Measure	How to evaluate
Science	Useful discoveries / analysed samples
Safety	No collision or equipment damage
Energy	Useful work per unit of battery
Navigation	Targets reached successfully
Communication	Important data transmitted
Mission	Percentage of planned objectives completed

v. Agent architecture

Best choice: model-based utility-based agent. The rover keeps an internal model because it cannot see the whole environment. It uses utility to compare science value, safety, energy and time. A simple reflex agent cannot make these longer-term trade-offs.

Example decision: Route A gives high science value but high risk; Route B gives lower science value but is safer. The agent calculates expected utility for both and chooses the higher-utility safe action.

QUESTION 3

8-Queens Problem – Step-by-Step Search

Goal: place 8 queens so no two share a row, column or diagonal. Represent a state as [r1,r2,...,r8], where ri is the row of the queen in column i.

A placement is valid only if: (1) row is unused, and (2) for every earlier queen j, |ri−rj| ≠ |i−j|.

Col.	Try row	Calculation / conflict check	Result
1	1	No previous queen → valid	Accept [1]
2	5	|5−1|=4; column gap=1 → 4≠1; row 5 unused	Accept [1,5]
3	8	vs C1: |8−1|=7≠2; vs C2: |8−5|=3≠1	Accept [1,5,8]
4	6	vs C1: 5≠3; C2: 1≠2; C3: 2≠1	Accept [1,5,8,6]
5	3	C1:2≠4; C2:2≠3; C3:5≠2; C4:3≠1	Accept [1,5,8,6,3]
6	7	C1:6≠5; C2:2≠4; C3:1≠3; C4:1≠2; C5:4≠1	Accept [1,5,8,6,3,7]
7	2	C1:1≠6; C2:3≠5; C3:6≠4; C4:4≠3; C5:1≠2; C6:5≠1	Accept [1,5,8,6,3,7,2]
8	4	C1:3≠7; C2:1≠6; C3:4≠5; C4:2≠4; C5:1≠3; C6:3≠2; C7:2≠1	Accept [1,5,8,6,3,7,2,4]

Final configuration = [1,5,8,6,3,7,2,4]. Each number is a row; its position is the column.

Final check	Calculation	Result
Rows	1,5,8,6,3,7,2,4 are all different	✓
Columns	One queen was placed in each column	✓
Diagonals	For every pair, |ri−rj| ≠ |i−j|	✓
Goal	8 queens placed with no attacks	✓

Therefore this configuration is a valid 8-Queens solution. Backtracking is appropriate because a conflicting placement can be rejected immediately and the previous column can be revisited.

QUESTION 4

OLA Cab Booking – Problem-Solving Agent and Pseudocode

Goal: select a suitable available cab from Mini, Micro, Sedan, Shared, Prime, etc. The best choice depends on fare, travel time, comfort and customer preference.

Agent type: Utility-based problem-solving agent.

Why: Several choices can reach the same destination. The agent must compare their benefits and costs, not merely find any route.

Illustrative utility calculation

Cab	Comfort C	Fare F	Time T	Example decision
Shared	2/5	■180	45 min	Cheap, low comfort
Micro	3/5	■220	35 min	Cheap + moderate comfort
Mini	3/5	■250	32 min	Balanced
Sedan	4/5	■350	30 min	More comfort
Prime	5/5	■500	28 min	Highest comfort, highest fare

For a numerical utility example, use: Utility = 0.5×ComfortScore + 0.3×CostScore + 0.2×TimeScore. CostScore and TimeScore are normalized by the application. The weights can change according to the customer's preference.

Step-by-step decision process

Step	Calculation / action
1	Input pickup and destination.
2	Get all available cab types and their fare, ETA, travel time and comfort.
3	Remove unavailable cabs.
4	Convert fare and time into preference scores.
5	Calculate Utility = 0.5C + 0.3CostScore + 0.2TimeScore.
6	Compare all utility values.
7	Select maximum utility.
8	Show cab, fare and ETA; confirm booking.

Pseudocode

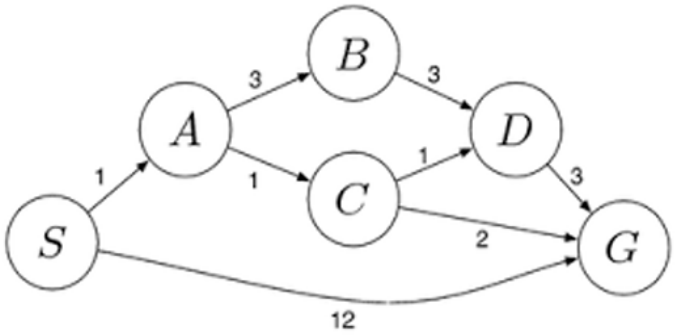
BEGIN
Read pickup, destination, preferences
Get available cabs
bestCab = NONE; bestUtility = –∞
FOR each cab
   Get fare, time, ETA, comfort
   IF unavailable: CONTINUE
   Calculate CostScore and TimeScore
   Utility = 0.5×Comfort + 0.3×CostScore + 0.2×TimeScore
   IF Utility > bestUtility: update bestCab and bestUtility
END FOR
IF bestCab exists: display and request confirmation
ELSE: display “No suitable cab available”
END

Final result: choose the available cab with the highest calculated utility. This satisfies the goal while considering the customer's comfort, cost and time.

QUESTION 5

Uniform Cost Search – Step-by-Step Calculation

Goal: find the minimum-cost path from S to G. UCS uses accumulated cost g(n) and always expands the lowest-cost node in the priority queue.



Edges: S→A=1, S→G=12, A→B=3, A→C=1, B→D=3, C→D=1, C→G=2, D→G=3.

Step 1: Expand S

New node	Calculation	Cost	Queue
A	g(S)+1 = 0+1	1	A(1)
G	g(S)+12 = 0+12	12	A(1), G(12)

Choose A because 1 < 12.

Step 2: Expand A

New node	Calculation	Cost	Queue
B	g(A)+3 = 1+3	4	C(2), B(4), G(12)
C	g(A)+1 = 1+1	2	C(2), B(4), G(12)

Choose C because 2 < 4 < 12.

Step 3: Expand C

New node	Calculation	Cost	Queue
D	g(C)+1 = 2+1	3	D(3), B(4), G(12)
G	g(C)+2 = 2+2	4	D(3), G(4), B(4)

Choose D because 3 is the smallest.

Step 4: Expand D

New path to G	Calculation	Cost	Comparison
D→G	g(D)+3 = 3+3	6	Existing G cost = 4, so keep 4

Step 5: Select G. Its current cost is 4 and all remaining frontier costs are ≥4, so UCS stops.

Complete route	Calculation	Total
S→G	12	12
S→A→B→D→G	1+3+3+3	10
S→A→C→D→G	1+1+1+3	6
S→A→C→G	1+1+2	4

Final answer: S → A → C → G. Minimum cost = 1+1+2 = 4.

Interpretation: Direct S→G costs 12, while the UCS route costs 4. Saving = 12–4 = 8 cost units. Because UCS expands the lowest accumulated cost and all edge costs are non-negative, the selected route is optimal.